

External Integrations

Integration principles

FinWallet treats every simulator as an external provider even though simulator source code lives in the same repository.

Rules:

- provider services are controller-based ASP.NET Core APIs;
- success/failure bodies use `ServiceResult<T>`;
- FinWallet never reads or writes provider storage directly;
- provider DTOs/enums stay behind Infrastructure anti-corruption adapters;
- internal and provider identifiers remain separate;
- external HTTP does not run while a financial SQL transaction is open;
- retries are used only when operation-level idempotency makes them safe;
- normal provider calls pass through YARP Gateway.

Gateway provider routes

```
/providers/bank/  
/providers/fraud/  
/providers/cutoff/  
/providers/campaign/  
/providers/communication/
```

Local default: `http://localhost:8080/providers/....`

FinWallet.Api calls the Gateway with `X-Internal-Service-Key`. Gateway validates that credential and uses a separate `DownstreamServiceKey` on the destination request. Provider business endpoints reject direct calls without the downstream key.

HttpClient policy

Provider adapters use `HttpClientFactory` + `SocketsHttpHandler` with:

- provider-specific timeout;
- max connections per server;
- pooled connection lifetime/idle timeout;
- GZip/Deflate/Brotli decompression;
- cookies disabled;
- internal-service-header delegating handler.

Arbitrary financial POSTs are not automatically retried because a timeout does not prove that the remote side effect did not happen.

FakeCommunication.Api

Simulates SMS/communication. Primary endpoint:

```
POST /api/v1/communication/sms
```

FinWallet route:

```
POST /providers/communication/api/v1/communication/sms
```

OTP bodies are sensitive and must not be logged in production. Fake modes can simulate failure, delay and timeout.

FakeBank.Api

Owns provider-side account/transaction state and never mutates FinWallet Wallet/Ledger tables.

Supported provider concepts include:

- account opening/read/activation/pending;
- deposit/withdrawal-like transaction start/finalize/read;
- statement data for reconciliation;
- duplicate protection through stable provider request keys.

IBankProvider is the Application port and FakeBankProvider the Infrastructure adapter. The adapter maps envelopes/enums/currency, propagates correlation and does not perform unsafe automatic financial retries.

FakeFraud.Api

Independent external-fraud simulator.

```
POST /providers/fraud/api/v1/fraud/evaluate
```

Requests contain server-derived risk signals. Raw device ID is replaced with a stable hashed reference. External fraud is mandatory for transfer when required; timeout/network/malformed responses fail closed.

FakeCutoff.Api

Simulates business-calendar/cutoff/settlement calculations by bank, currency and transaction type. It is mainly planned for BankDeposit and BankWithdrawal. Simulated holiday data is not a legal production source.

FakeCampaign.Api

May calculate campaign eligibility, discount and sponsor identity. FinWallet remains responsible for representing the economic effect in a balanced ledger.

Health and load balancing

YARP clusters use configuration-driven destinations and PowerOfTwoChoices. Critical provider clusters use active health checks; the main FinWallet cluster also has passive transport-failure health handling.

SSRF / unsafe upstream prevention

Clients cannot provide arbitrary provider URLs. Provider destinations are server-owned YARP/config values, so normal request payloads cannot turn the application into an HTTP client for user-controlled URLs.

Production network rule

Application-level downstream keys reduce direct-bypass risk, but production backend/provider services should additionally be non-publicly-routable through network policy/ingress controls.